

Recursion vs. Iteration in Java :

Performance and Scalability Across Four Algorithms

Ryan Merchant, Carter McLean, Addison Lin, and Sebastian Larimore

University of Oklahoma - Computer Science | CS2334 Final Project, Spring 2026

Abstract

This short research paper expands the poster into a readable handout for presentation attendees, and we as a team thank you for your visit to the OU CS Research Day. The project compares recursive and iterative Java implementations of Fibonacci sequence, factorial, binary search, and fast exponentiation. The central question is when recursion becomes less practical than iteration because of call-stack growth, function-call overhead, or `StackOverflowError` failures. Each algorithm was tested at input sizes from 5,000 through 100,000, with 10 runs averaged using `System.nanoTime()`. Results show that iteration is the safer default for large inputs, but the reason is not simply that loops are always faster. The deciding factor is recursion depth. Fibonacci and factorial used linear-depth recursion and overflowed at 50,000. Binary search and fast exponentiation used logarithmic-depth recursion and remained safe through 100,000.

1. Introduction and Background

Recursion and iteration are two ways to express repeated computation. Recursion solves a problem by calling the same method on a smaller subproblem, while iteration repeats a block of code with a loop. In small examples, recursion can be easier to read because it often resembles the mathematical definition of the problem. In production-style Java code, however, every recursive call creates another frame on the JVM call stack. Oracle describes `StackOverflowError` as the error thrown when an application recurses too deeply, so overflow is a practical reliability issue rather than only a theoretical concern [1].

This distinction matters because two algorithms can both be recursive but have very different stack behavior. A linearly recursive algorithm, such as a simple recursive factorial, adds roughly one call per input step. A logarithmic-depth recursive algorithm, such as binary search or exponentiation by squaring, cuts the problem down much faster. Therefore, this paper focuses on a more precise claim: iteration tends to be safer at large input sizes, but recursion can remain practical when its depth is bounded and small.

2. Methodology

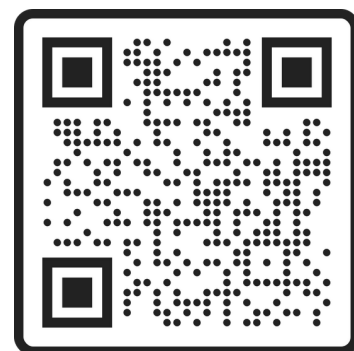
The study implemented both recursive and iterative versions of four algorithms: Fibonacci sequence, factorial, binary search, and fast exponentiation. The benchmark inputs were 5,000, 10,000, 30,000, 50,000, 70,000, and 100,000. Each test used Java timing with `System.nanoTime()`, which Oracle documents as a high-resolution time source intended for elapsed-time measurement [2]. Each benchmark was run 10 times and averaged, and recursive crashes were recorded as part of the result rather than ignored. This made the experiment measure both speed and reliability.

Algorithm	Recursive depth pattern	Largest safe recursive input	First recursive overflow
Fibonacci Sequence	Linear	30,000	50,000
Factorial	Linear	30,000	50,000
Binary Search	Logarithmic	100,000	None observed
Fast Exponentiation	Logarithmic	100,000	None observed

Table 1. The main reliability result: the two linearly recursive algorithms overflowed at 50,000, while the logarithmic-depth algorithms did not overflow in the tested range.



Raw Benchmark Data Direct to PDF



GitHub Repo Link to Base Code

3. Results

The results show a clear recursion cliff for Fibonacci and factorial. Both recursive versions completed through 30,000 but overflowed at 50,000. Their iterative versions continued through the full tested range. Binary search stayed safe in both forms, but the iterative implementation was faster across the tested inputs. Fast exponentiation was the strongest example where recursion stayed competitive, because its recursive structure reduces the exponent quickly instead of making one call per input step.

Recursive vs Iterative Runtime by Algorithm

Average runtime in nanoseconds across the tested input sizes. Overflow points are marked with X.

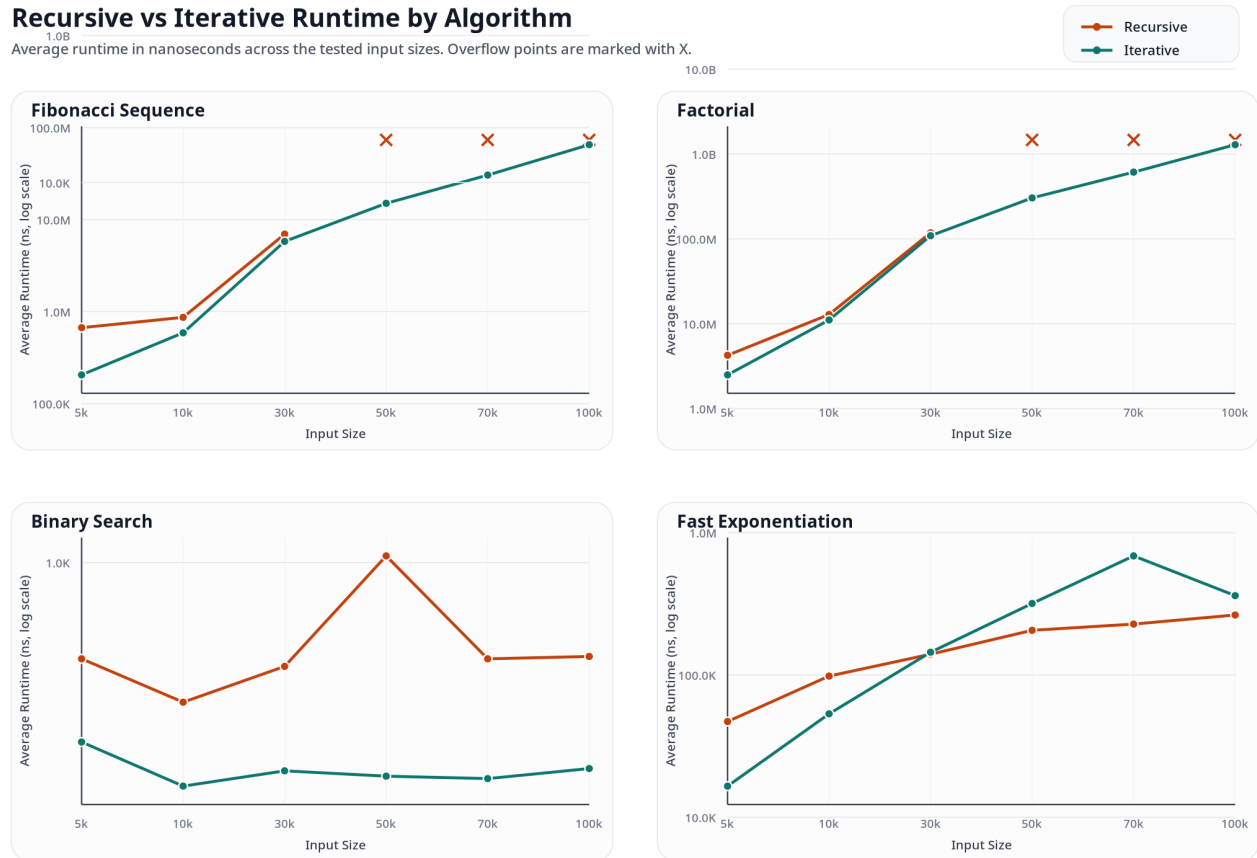


Figure 1. Selected as the main evidence figure because it shows all four algorithms, their runtime trends, and the overflow points in one view.

3.1 What the Results Mean

- Fibonacci and factorial show why linear recursion can become unsafe: increasing the input increases the call-stack depth almost directly.
- Binary search shows that recursion can be safe when depth is logarithmic, but loop-based code may still avoid method-call overhead.
- Fast exponentiation shows that recursion should not be rejected automatically; when the recursive depth is small, recursive code can remain practical and sometimes faster in a specific implementation.
- The binary-search spike should be interpreted carefully. It may reflect runtime noise, JVM behavior, or the benchmark setup rather than the algorithm alone.

4. Discussion

The most important result is that algorithm structure matters more than the word recursive by itself. Recursive Fibonacci and factorial are risky at large input sizes because their depth grows linearly. Binary search and fast exponentiation avoid that cliff because each call reduces the remaining work much faster. This explains why the study found both outcomes: iteration was generally more scalable, but shallow recursion could still be competitive. The experiment also has benchmarking limitations. Modern Java performance is affected by the HotSpot virtual machine, including just-in-time compilation and other runtime optimizations [3]. Simple `System.nanoTime()` timing is appropriate for a course project and for producing understandable poster evidence, but stronger microbenchmarking would normally use warm-up runs, controlled JVM settings, and a framework such as OpenJDK JMH. The OpenJDK JMH project recommends a standalone benchmark setup for more reliable initialization and results, and Oracle warns that naive JVM benchmarks can be misleading without careful design [4], [5].

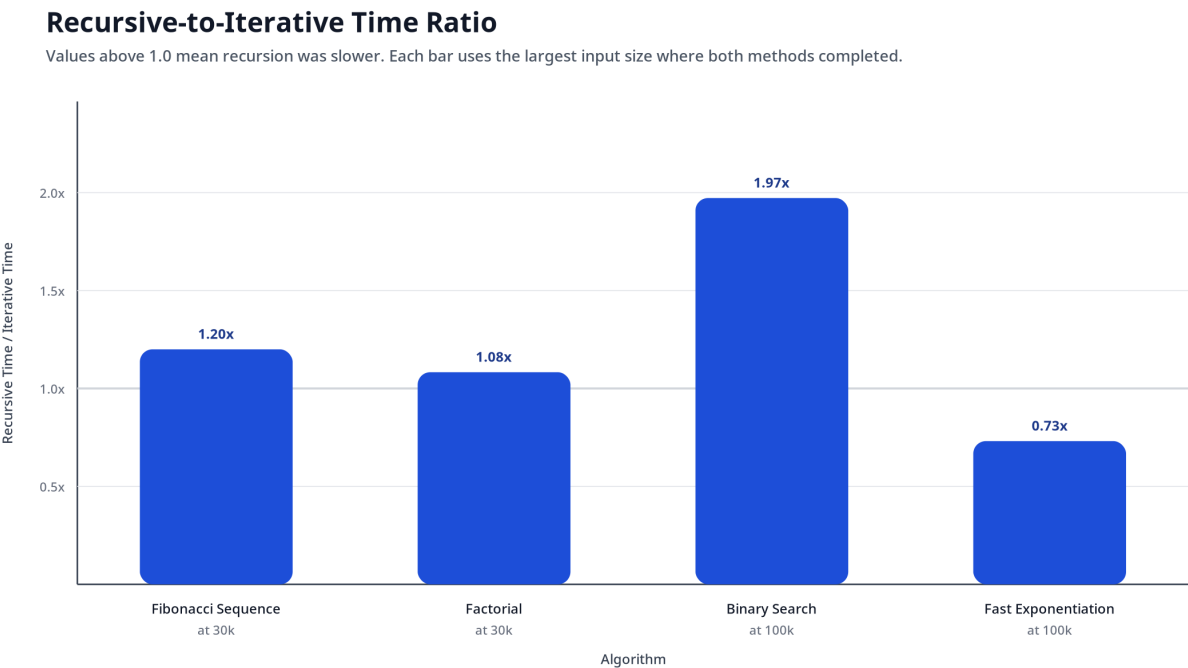


Figure 2. Selected as the conclusion figure because it compresses the practical runtime comparison into one ratio per algorithm. Values above 1.0 mean recursion was slower.

5. Conclusion

For this project, iteration was the safest general choice as input size increased. It avoided the stack overflow failures seen in recursive Fibonacci and factorial and was consistently faster for binary search. However, the study does not prove that recursion is always worse. Fast exponentiation demonstrates that recursion can remain efficient when the recursion depth is logarithmic. A practical rule is to use recursion when it makes the code clearer and the maximum depth is small or well understood; use iteration when input size can grow large, when every input step adds another recursive call, or when reliability is more important than matching a mathematical definition.

Future Work

A stronger follow-up study could rerun the benchmarks with JMH, add JVM warm-up trials, report variance rather than only averages, test multiple machines, profile memory and stack depth directly, and compare additional algorithms such as tree traversal, merge sort, and dynamic programming versions of Fibonacci.

References

Oracle. (n.d.). Class StackOverflowError. Java SE API Specification. Used to define the observed stack overflow failure condition.

Oracle. (n.d.). `System.nanoTime()`. Java SE API Specification. Used to describe elapsed-time measurement in the benchmark method.

Oracle. (n.d.). Java HotSpot Virtual Machine performance enhancements. Used to explain why JVM behavior can affect timing results.

OpenJDK. (n.d.). Java Microbenchmark Harness (JMH). Used as a future-work benchmark methodology recommendation.

Oracle. (2014). Avoiding benchmarking pitfalls on the JVM. Used to explain why simple timing loops should be interpreted carefully.

Liu, Y. A., & Stoller, S. D. (1999). From recursion to iteration. <https://doi.org/10.1145/328690.328700>

Mehta, M. K., Krynski, S., Gualandi, H. M., Thakur, M., & Vitek, J. (2023). Reusing Just-in-Time Compiled Code. Proceedings of the ACM on Programming Languages, 7(OOPSLA2), 1176-1197. <https://doi.org/10.1145/3622839>